

# Sistema Igreja

Projeto de recepcao e integracao de visitantes em celulas, com aplicativo Flutter e API REST em Node.js/TypeScript.

## Perfis de usuario

Sistema multi-tenant: cada igreja (**churchId**) so enxerga seus proprios dados (isolamento via tenant-guard no Prisma). O unico perfil sem igreja fixa e o **SUPERADMIN**, dono do SaaS, que opera cross-tenant.

Usuarios reais abaixo, um por perfil (base local de desenvolvimento). Senha exibida em texto puro apenas por serem contas de teste/seed — nunca faca isso com dados de producao.

| Perfil      | Funcionalidades                                                                                                                                                                                                                                                | Igreja (tenant)                                                                   | Email                         | Senha         |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------|---------------|
| SUPERADMIN  | Acesso irrestrito a todas as rotas; gestao de igrejas, planos e billing do SaaS; passa em toda checagem de admin/staff/feature-gate.                                                                                                                           | Nenhuma fixa — cross-tenant ( <b>churchId = null</b> ), enxerga todas as igrejas. | superadmin@sistema.local      | superadmin123 |
| ADMIN       | CRUD de celulas; gestao de membros e materiais; conversao de visitante em membro; dashboard com indicadores; passa como staff, supervisor e equipe kids/responsavel para depuracao.                                                                            | Igreja Principal ( <b>00000000-0000-0000-0000-000000000001</b> )                  | admin@sistemaigreja.com.br    | admin123      |
| COORDENADOR | Gestao de coordenacao (supervisores e lideres vinculados); acesso de "equipe de lideranca" ( <b>requireStaff</b> ); feature <b>coordenacao</b> (depende do plano da igreja).                                                                                   | Igreja Principal ( <b>00000000-0000-0000-0000-000000000001</b> )                  | coordenador1@teste.igreja.com | senha123      |
| SUPERVISOR  | Acompanhamento dos lideres/celulas sob sua coordenacao; acesso de "equipe de lideranca" ( <b>requireStaff</b> ).                                                                                                                                               | Igreja Principal ( <b>00000000-0000-0000-0000-000000000001</b> )                  | supervisor1@teste.igreja.com  | senha123      |
| LIDER       | Lista e detalhe de visitantes com busca/filtro; alteracao de status; conversao de visitante em membro; registro e consulta de presenca por celula/data; historico espiritual dos visitantes; listagem/download de materiais da celula; perfil e foto proprios. | Igreja Principal ( <b>00000000-0000-0000-0000-000000000001</b> )                  | lider1@teste.igreja.com       | senha123      |
| KIDS        | Equipe da salinha infantil: check-in/check-out de criancas por QR ou senha, anotacoes de aula e alertas                                                                                                                                                        | Igreja Principal ( <b>00000000-0000-0000-</b>                                     | cris.kids@teste.com           | kids123       |

| Perfil      | Funcionalidades                                                                                                                               | Igreja (tenant)                                                        | Email               | Senha  |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|---------------------|--------|
|             | aos responsaveis — restrito as salas em que e professor ( <code>makeRequireRoomAccess</code> ); feature <code>kids</code> (depende do plano). | 0000–<br>000000000001)                                                 |                     |        |
| RESPONSAVEL | So visualiza os proprios filhos: acompanha check-in/check-out e recebe alertas da salinha; nao faz parte da equipe de lideranca.              | Igreja Principal<br>(00000000–<br>0000–0000–<br>0000–<br>000000000001) | maria.mae@teste.com | mae123 |

## Documentacao em formato Book

Para manter o README enxuto, toda a documentacao detalhada foi organizada em capitulos:

- [Book de Documentacao](#)
- [Summary do Book](#)

Conteudo incluido no book:

- Funcionalidades por perfil (Admin, Lider e fluxo de Visitante)
- Mapa completo da API REST
- Arquitetura frontend e backend
- Diagramas de DER, Sequencia e Classes
- Galeria com imagens da identidade visual do app
- Guia de execucao e operacao

## Execucao rapida

### App Flutter

```
flutter pub get
flutter run
```

### API

```
cd api
npm install
npx prisma generate
npx prisma migrate dev
npm run dev
```

## Publicar PWA (App web)

Esta base contém um fluxo para gerar o build do Flutter Web localmente, pré-comprimir os ativos e empacotar em uma imagem Docker multi-arquitetura pronta para push.

- Arquivos importantes:
  - `api/package.json` — script `publishAPP` que executa o fluxo localmente.
  - `scripts/precompress_web.sh` — gera `.gz` e `.br` em `build/web`.
  - `Dockerfile.web` — Dockerfile que serve o conteúdo de `build/web`.

1. Testar localmente (requer `flutter`, `docker buildx` e `brotli` instalados):

```
# via npm (args passados diretamente ao script publish_app.sh)
npm --prefix api run publishAPP -- --tag=0.0.2 --api_base_url="https://minha.api/v1"

# ou via variáveis de ambiente
TAG=0.0.2 API_BASE_URL="https://minha.api/v1" npm --prefix api run publishAPP

# ou direto pelo script
bash scripts/publish_app.sh --tag=0.0.2 --api_base_url="https://minha.api/v1"
```

#### Observações:

- `--tag` define a tag da imagem Docker; padrão: `latest`.
- `--api_base_url` define a URL base da API embutida no build; padrão: placeholder substituído em runtime pelo `entrypoint.sh`.

#### 2. CI (GitHub Actions)

Um workflow está disponível em `.github/workflows/publish_app.yml`. Ele:

- Faz checkout do código;
- Configura Flutter (runner nativo) e executa `flutter build web` com `--dart-define=API_BASE_URL`;
- Executa `./scripts/precompress_web.sh build/web`;
- Faz login no Docker Hub e publica a imagem multi-arch usando a `tag` do push (refs/tags/v\*) ou o `tag` fornecido manualmente.

Secrets necessários no repositório para o workflow:

- `DOCKERHUB_USERNAME` — usuário Docker Hub;
- `DOCKERHUB_TOKEN` — token/password para login no Docker Hub;
- `API_BASE_URL` — (opcional) URL base da API usada no build se não fornecida manualmente.

Uso do workflow:

- Push de uma tag `vX.Y.Z` aciona a publicação usando `X.Y.Z` como tag da imagem.
- Ou faça `Actions -> Publish Flutter Web App -> Run workflow` e informe `tag` e `api_base_url`.